

INSTITUCIÓN:

Universidad Provincial de Oficios 'UPRO'

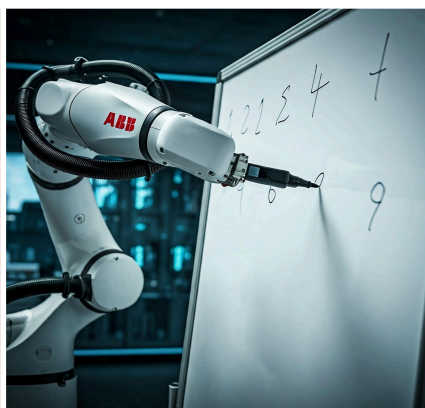
CARRERA:

Tecnicatura en robótica con orientación en inteligencia artificial

Fecha: 06/12/2024

TRABAJO FINAL

Desarrollo de un sistema de escritura robótica basado en visión por computadora utilizando redes neuronales convolucionales



Integrantes:

Uriel Ravelli

Rodrigo Murua

Manuel Sosa

Sebastian Leguizamon

1. Objetivo del desafío

Crear un prototipo funcional de una celda robotizada utilizando el robot ABB IRB 1110 y algoritmos de visión por computadora con inteligencia artificial para reconocer números manuscritos captados por una cámara y dibujarlos en un pizarrón.

- El proceso debe iniciarse mediante un pulsador físico.
- Deberá existir una parada de emergencia que detenga todo el proceso.
- Crear un soporte para que la cámara pueda colocarse al brazo del robot.

2. Introducción

Este proyecto propone un sistema de visión por computadora basado en deep learning para el reconocimiento de dígitos numéricos escritos a mano. Utilizando **redes neuronales convolucionales**, el sistema procesa imágenes capturadas por una cámara para identificar los dígitos escritos con un marcador. Posteriormente, un brazo robótico replica los dígitos reconocidos en la misma pizarra, demostrando la viabilidad de una interacción humano-máquina en tiempo real basada en la escritura a mano.

> **El proceso funciona de la siguiente manera:**

1. Dibujar cualquier número dentro del cuadro marcado en el pizarrón.
2. Presionar el botón de inicio.
3. El robot irá al punto de lectura y leerá el número con la cámara.
4. Se enviará una señal a python para que el modelo realice la predicción.
5. Luego python enviará la señal con la predicción al controlador.
6. El robot dibuja el número predicho y comienza el ciclo de nuevo.

3. Marco teórico

¿Qué es la Inteligencia Artificial?

La **Inteligencia artificial (IA)**, en el contexto de las ciencias de la computación, es una disciplina y un conjunto de capacidades cognitivas e intelectuales expresadas por sistemas informáticos o combinaciones de algoritmos cuyo propósito es la creación de máquinas que imiten la inteligencia humana para realizar tareas, y que pueden mejorar conforme recopilen información.

¿Cómo funciona?

La IA se basa en algoritmos y modelos matemáticos que permiten a las máquinas aprender de los datos. Los **datos** son la clave para entrenar a los modelos de IA. A través del aprendizaje, los modelos pueden identificar patrones, hacer predicciones y tomar decisiones.

Se puede clasificar según su nivel de inteligencia:

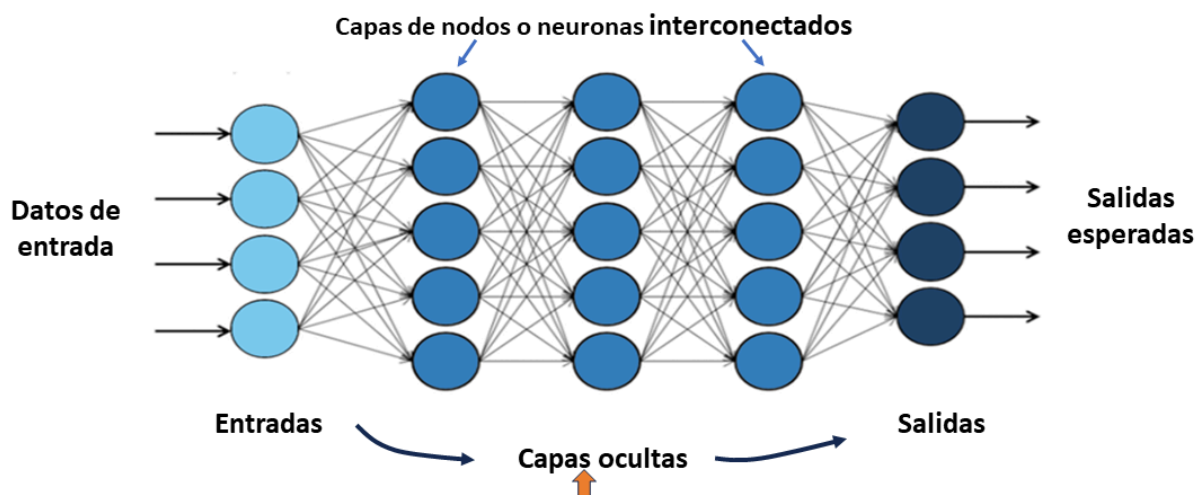
- > **Inteligencia Artificial Estrecha (IA débil):** Realiza tareas específicas y no posee conciencia ni comprensión general del mundo.
- > **Inteligencia Artificial General (IA fuerte):** Esta IA se asemeja a la inteligencia humana, es capaz de aprender, entender, razonar y aplicar su conocimiento a una amplia variedad de tareas.

Aprendizaje Automático

El **aprendizaje automático** (AA) del inglés machine learning, “ML”, es el subcampo de las ciencias de la computación y una rama de la inteligencia artificial, cuyo objetivo es desarrollar técnicas que permitan que las computadoras aprendan. Se dice que un agente aprende cuando su desempeño mejora con la experiencia y mediante el uso de datos; es decir, cuando la habilidad no estaba presente en su genotipo o rasgos de nacimiento. En el aprendizaje de máquinas un computador observa datos, construye un modelo basado en esos datos y utiliza ese modelo a la vez como una hipótesis acerca del mundo y una pieza de software que puede resolver problemas.

Redes neuronales artificiales

En el aprendizaje automático, una **red neuronal artificial** (abreviada ANN o NN) es un modelo dentro de los llamados sistemas conexionistas, inspirado en la estructura y función de las redes neuronales biológicas en los cerebros animales. Una ANN consta de unidades o nodos conectados llamados neuronas artificiales, que modelan vagamente las neuronas del cerebro. Estas están conectadas por bordes, que modelan las sinapsis del cerebro. Cada neurona artificial recibe «señales» de las neuronas conectadas, luego las procesa y envía una señal a otras neuronas conectadas. La «señal» es un número real, y la salida de cada neurona se calcula mediante una función no lineal de la suma de sus entradas, llamada función de activación. La fuerza de la señal en cada conexión está determinada por un peso, que se ajusta durante el proceso de aprendizaje.



Por lo general, las neuronas se agrupan en capas. Las diferentes capas pueden realizar diferentes transformaciones en sus entradas. Las señales viajan desde la primera capa (la capa de entrada) hasta la última capa (la capa de salida), posiblemente pasando por múltiples capas intermedias (capas ocultas). Una red se denomina típicamente red neuronal profunda si tiene al menos dos capas ocultas.

Visión por computadora

En el campo de la inteligencia artificial la **visión por computadora** es una disciplina científica que incluye métodos para adquirir, procesar, analizar y comprender las imágenes del mundo real con el fin de producir información numérica o simbólica para que puedan ser tratados por un ordenador y tenga la misma capacidad de "**ver**" y **comprender** el mundo visual de la misma manera que lo hacemos los humanos.

¿Cómo funciona en términos básicos?

> **Adquisición de imágenes:** Comienza con una cámara digital que captura una escena del mundo real y la convierte en una imagen digital. Esta imagen está compuesta por una matriz de números, donde cada número representa un píxel con un valor de intensidad (en escala de grises) o de color (RGB).

> **Preprocesamiento de imágenes:** La imagen capturada se somete a una serie de operaciones para mejorar su calidad y facilitar su análisis. Esto puede incluir:

- > **Reducción de ruido:** Eliminar elementos aleatorios que puedan interferir en el análisis.
- > **Segmentación:** Dividir la imagen en regiones de interés.
- > **Realce de bordes:** Identificar los límites entre objetos.

> **Extracción de características:** Una vez preprocesada, la imagen se analiza para extraer características relevantes. Estas características pueden ser:

- > **Bordes:** Líneas que delimitan objetos.
- > **Texturas:** Patrones repetitivos en la imagen.
- > **Formas:** Contornos y geometrías de los objetos.
- > **Colores:** Tonalidades presentes en la imagen.

> **Reconocimiento de patrones:** Estas características extraídas se comparan con patrones conocidos para identificar objetos, caras, escenas o cualquier otro elemento de interés. Esta etapa suele involucrar el uso de algoritmos de aprendizaje automático, como las redes neuronales convolucionales (CNN), que son especialmente eficaces en tareas de visión por computadora.

> **Toma de decisiones:** Basándose en la información extraída y los patrones reconocidos, la computadora puede tomar decisiones o realizar acciones específicas. Por ejemplo, puede clasificar una imagen como una foto de un gato, identificar una cara en una multitud o controlar un robot para seguir una línea.

En resumen, la visión por computadora es un campo interdisciplinario que combina la informática, las matemáticas y la ingeniería para permitir que las computadoras interpreten el mundo visual.

Redes Neuronales Convolucionales

Las **Convolutional Neural Network (CNN)** son un tipo específico de redes neuronales artificiales diseñadas para trabajar con datos visuales, como imágenes. Son especialmente eficientes para tareas como el reconocimiento de objetos, la segmentación de imágenes y, en nuestro caso, la clasificación de dígitos escritos a mano.

Su arquitectura se inspira en la organización del córtex visual del cerebro, lo que les permite extraer características de las imágenes de manera jerárquica.

¿Por qué las CNN son ideales para la clasificación de imágenes?

> **Extracción de características:** Las CNN están diseñadas para aprender automáticamente las características más relevantes de una imagen, como bordes, texturas y formas. Esto les permite discriminar entre diferentes clases de objetos.

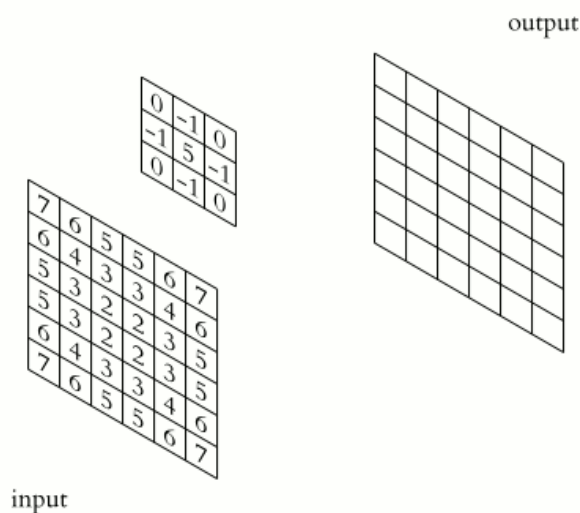
> **Invariabilidad a traslaciones, rotaciones y escalas:** Las CNN son capaces de reconocer un objeto independientemente de su posición, orientación o tamaño en la imagen.

> **Jerarquía de características:** Las capas convolucionales sucesivas construyen representaciones cada vez más abstractas de la imagen, desde características locales hasta conceptos de alto nivel.

Las CNN están compuestas por varias capas:

- > **Capa convolucional:** Capa principal que aplica filtros a la imagen de entrada para extraer características como bordes, texturas y formas.
- > **Capa de agrupamiento:** Reduce la dimensionalidad de los datos y ayuda a la red a ser invariante a pequeñas traslaciones.
- > **Capas completamente conectadas:** Estas capas realizan la clasificación final de la imagen, similar a una red neuronal tradicional.

Funcionamiento de las redes neuronales convolucionales



4. Desarrollo

ENTORNO DE DESARROLLO

Entorno de desarrollo integrado (IDE):

- [Visual studio code](#)

Lenguajes de programación:

- [Python](#)
- [RAPID](#) (Utilizando RobotStudio)

Librerías:

- [TensorFlow](#): Para construir y entrenar modelos de aprendizaje profundo.
- [Keras](#): API de alto nivel construida sobre TensorFlow que simplifica la creación de modelos. (Se incluye al instalar TensorFlow 2.0 en adelante)
- [OpenCV](#): Para procesamiento de imágenes.
- [Numpy](#): Para trabajar con arrays.

MODELO DE IA

Entrenar un modelo de inteligencia artificial para que reconozca dígitos escritos a mano sin importar el tamaño, posición, rotación y estilo es algo complejo y sin dudas un gran desafío. Por eso, abordaremos paso a paso sobre cómo crear un modelo suficientemente capaz de realizar esta tarea.

Crear modelo paso a paso

Para seguir los pasos que veremos a continuación deberán tener instalado Visual Studio Code y python junto a las librerías mencionadas anteriormente. Una vez hecho, debes crear un módulo al que llamaremos “modelo” con la extensión **.py**. Si quieres evitar instalar todo lo mencionado puedes usar [Google Colab](#).

1. Importar librerías

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import TensorBoard
```

2. Cargar dataset

Cargamos conjunto de datos de [Mnist](#) que contiene imágenes ya etiquetadas de números escritos a mano. (60.000 para entrenamiento y 10.000 para pruebas)

```
(X_entrenamiento, Y_entrenamiento), (X_pruebas, Y_pruebas) =
mnist.load_data()
```

3. Reformar los datos

Establecer un shape a los datos que la red espera. El índice 0 del shape de los datos es igual a la cantidad de imágenes que estos contienen. 28, 28 son las dimensiones de las imágenes en píxeles y por último, el 1 es el canal (escala de grises). $X_{entrenamiento} = (60000, 28, 28, 1)$ $X_{pruebas} = (10000, 28, 28, 1)$

```
X_entrenamiento = X_entrenamiento.reshape(X_entrenamiento.shape[0],
28, 28, 1)
X_pruebas = X_pruebas.reshape(X_pruebas.shape[0], 28, 28, 1)
```

4. Hacer 'one-hot encoding'

Se transforma cada etiqueta numérica (dígito) en un vector donde solo un elemento es 1 (indicando la clase correcta) y el resto son 0. Por ejemplo, el número '3' se convierte en [0, 0, 0, 1, 0, 0, 0, 0, 0, 0].

```
Y_entrenamiento = to_categorical(Y_entrenamiento)
Y_pruebas = to_categorical(Y_pruebas)
```

5. Normalizar los datos

Los valores de los píxeles se escalan entre 0 y 1 para mejorar el entrenamiento.

```
X_entrenamiento = np.array(X_entrenamiento).astype(float) / 255
X_pruebas = np.array(X_pruebas).astype(float) / 255
```

6. Aumento de datos

Configurar un generador de imágenes que aplicará transformaciones aleatorias a partir de las imágenes originales, creando nuevas imágenes. Esto permitirá aumentar el tamaño y la diversidad del conjunto de datos, lo que a su vez mejorará el rendimiento del modelo.

```
datagen = ImageDataGenerator(
    rotation_range = 30,
    width_shift_range = 0.25,
    height_shift_range = 0.25,
    zoom_range=[0.5,1.5],
)
datagen.fit(X_entrenamiento)
```

En pocas palabras, este código "enseña" al generador de datos cómo son las imágenes de entrenamiento para que pueda aplicar transformaciones realistas y útiles.

7. Preparar los datos para el entrenamiento

Creamos un generador de datos (data_gen) que suministrará lotes de imágenes transformadas aleatoriamente para entrenar el modelo.

```
data_gen_entrenamiento = datagen.flow(X_entrenamiento, Y_entrenamiento,
batch_size=32)
```

8. Definir la arquitectura de la red

Creamos un modelo secuencial, lo que significa que la salida de una capa se convierte en la entrada de la siguiente.

```
modelo = tf.keras.models.Sequential([
    # Primera capa convolucional con 32 filtros con un kernel de 3x3 y
    # funcion de activacion ReLU
    tf.keras.layers.Conv2D(32, (3,3),
                           activation='relu',
                           input_shape=(28, 28, 1)),

    # Primera capa de pooling que reduce la dimensionalidad espacial de
    # la imagen a la mitad conservando las características más importantes
    tf.keras.layers.MaxPooling2D(2, 2),

    # Segunda capa convolucional con 64 filtros que extraen
    # características aún más complejas y abstractas.
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),

    # Segunda capa de pooling: Reduce aún más la dimensionalidad
    tf.keras.layers.MaxPooling2D(2,2),

    # Capa de dropout: Evita el sobreajuste al desactivar aleatoriamente
    # un 50% de las neuronas durante el entrenamiento
    tf.keras.layers.Dropout(0.5),

    # Aplana la salida de la capa convolucional: Convierte la matriz
    # resultante de la convolución en un vector para poder conectarla a las
    # capas densamente conectadas.
    tf.keras.layers.Flatten(),

    # Capa densamente conectada con 100 neuronas que realizan una
    # clasificación inicial de las características.
    tf.keras.layers.Dense(100, activation='relu'),

    # CAPA DE SALIDA con 10 neuronas: Cada neurona representa una clase.
    # Función de activación SOFTMAX que produce una distribución de
    # probabilidad sobre las 10 clases, indicando la probabilidad de que la
    # imagen pertenezca a cada una de ellas.
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Esta red neuronal "aprende a ver" y a categorizar imágenes de la misma manera que un humano podría hacerlo. Examina las imágenes en busca de características distintivas y luego decide a qué categoría pertenecen.

9. Compilar el modelo

Configurar el modelo para que se entrene utilizando el algoritmo Adam, minimizando la pérdida de entropía cruzada categórica y monitoreando la precisión durante el proceso de entrenamiento.

```
modelo.compile(optimizer='adam',  
               loss='categorical_crossentropy',  
               metrics=['accuracy'])
```

10. Crear directorio y callback para guardar registros del entrenamiento

TensorBoard es una herramienta de visualización potente que nos permite inspeccionar y comprender mejor el progreso del entrenamiento del modelo.

```
# Directorio donde se guardarán los logs  
log_dir = "logs/fit/registro_entrenamiento"  
# Crear el callback de TensorBoard  
tensorboard_callback = TensorBoard(log_dir=log_dir)
```

11. Entrenar el modelo

```
lote = 32 # Tamaño de lote de datos que se procesaran en cada iteración  
epocas= 100 # Número de pasadas completas por el conjunto de  
entrenamiento  
  
historial = modelo.fit(  
    data_gen_entrenamiento, # Datos de entrenamiento  
    epochs=epocas,  
    batch_size=lote,  
    validation_data=(X_pruebas, Y_pruebas), # Datos de validación  
    steps_per_epoch=int(np.ceil(60000 / float(lote))),  
    validation_steps=int(np.ceil(10000 / float(lote))),  
    callbacks=[tensorboard_callback])
```

12. Evaluar modelo

```
# Evaluar el modelo utilizando los datos de pruebas  
test_loss, test_acc = modelo.evaluate(X_pruebas, Y_pruebas)  
print('Pérdida:', test_loss)  
print('Precision:', test_acc)  
313/313 ————— 3s 8ms/step - accuracy:  
0.9866 -  
loss: 0.0454  
Pérdida: 0.03992597013711929  
Precision: 0.9879999756813049
```

El modelo ha mostrado un desempeño excepcional en el conjunto de prueba, lo que indica que es capaz de realizar predicciones precisas en datos no vistos anteriormente.

13. Guardar el modelo entrenado

Una vez que el modelo haya terminado el entrenamiento guardaremos el modelo en formato **.keras** o en **.h5** para posteriormente utilizarlo.
(El modelo se guardará en la carpeta donde estés trabajando)

```
modelo.save('modelo_entrenado.keras')
```

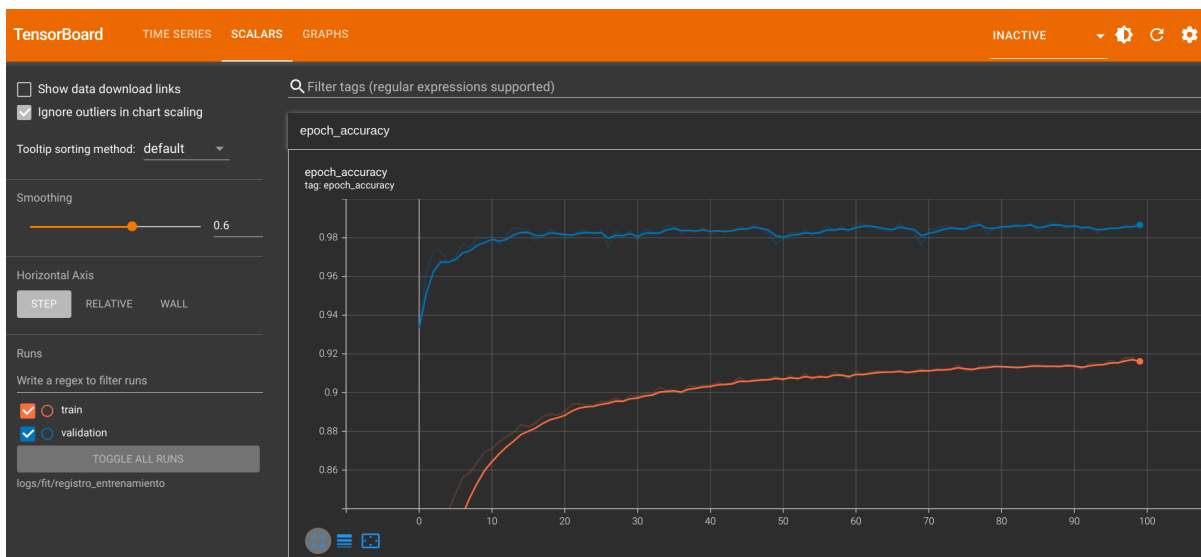
14. Ver registros del entrenamiento

Para iniciar TensorBoard y ver los graficos del progreso del entrenamiento debemos ejecutar el siguiente comando desde la terminal de tu computadora.

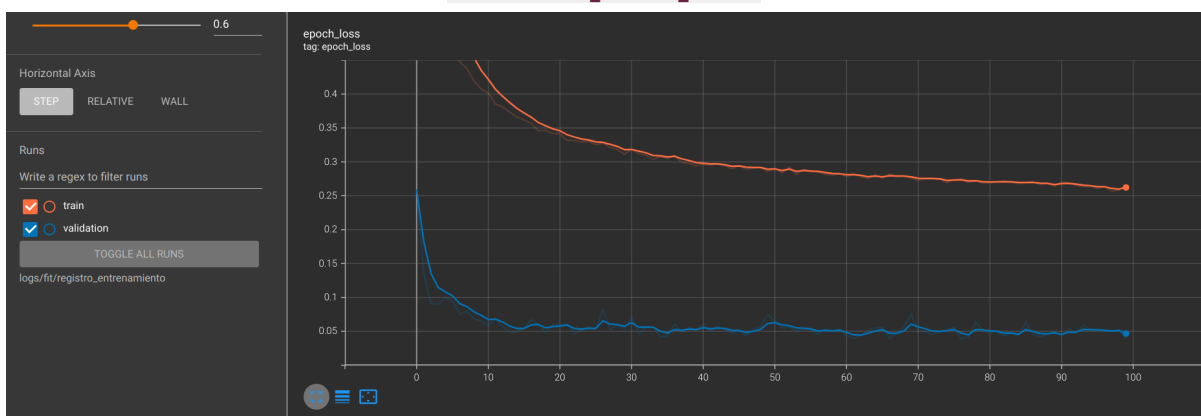
tensorboard --logdir “directorio donde se encuentra el log”

Luego debes abrir tu navegador y pegar la dirección <http://localhost:6006/>, allí encontrarás una interfaz gráfica interactiva donde podrás explorar los datos de tu entrenamiento.

Precisión por época



Pérdida por época



Como podemos ver en la época 15 o 20 los datos de pruebas alcanzan un buen nivel de precisión y de pérdida, pero podríamos decir que las épocas adecuadas para que los datos de entrenamiento mejoren aún un poco más son aproximadamente 50 epochs.

5. Hacer predicciones

A continuación veremos cómo preprocesar las imágenes captadas por la cámara para que el modelo pueda predecir los números sin ningún problema.

Para eso crearemos un nuevo módulo que se llamará **funciones.py** en el mismo directorio donde tengas el modelo entrenado.

> Importamos librerías necesarias y cargamos el modelo

```
import cv2
import tensorflow as tf
import numpy as np
from skimage.filters import threshold_otsu # "Se debe instalar skimage"

modelo = tf.keras.models.load_model("modelo_entrenado.keras")
```

> Creamos una función para preprocesar la imagen

```
def preprocesar_imagen(img):
    # Convertir a escala de grises
    img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Redimensionar la imagen a 28 x 28 píxeles
    img = cv2.resize(img, (28,28))
    # Binarización adaptativa
    thresh = threshold_otsu(img)
    img = cv2.threshold(img, thresh, 255, cv2.THRESH_BINARY_INV)[1]
    # Aplicar el filtro de Gaussiana
    img = cv2.GaussianBlur(img, (3,3), 0)
    # Normalizar la imagen
    img = img.astype(np.float32) / 255.0
    # Expandir dimensiones (1, 28, 28, 1)
    img = np.expand_dims(img, axis=0)
    img = np.expand_dims(img, axis=3)
    return img
```

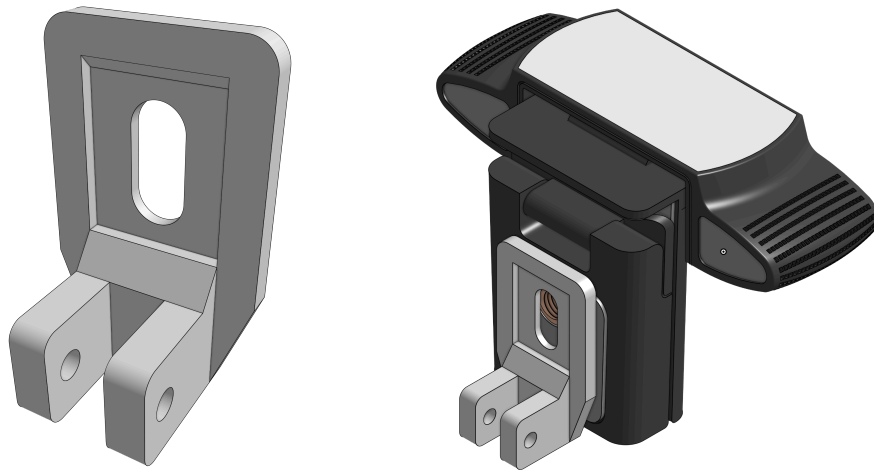
> Por último creamos una función para que el modelo haga las predicciones

```
def predecir_numero(img):
    # Hacer la predicción
    prediccion = modelo.predict(img)
    # Obtener el índice con mayor probabilidad
    prediccion = np.argmax(prediccion)
    # Guardamos la predicción en un número entero
    prediccion = int(prediccion)
    return prediccion
```

6. Componentes utilizados

- Robot ABB IRB 1100 - Controlador Omnicore C30.
- Cámara Logitech C920s PRO.
- Computadora portátil HP.
- Cable ethernet para la conexión al controlador del robot.
- Pulsador físico para inicio y parada de emergencia física.
- Soporte para el fibrón acoplable al robot (tool marker).
- Soporte diseñado en 3D para acoplar la cámara al brazo del robot.
- Fibrón y pizarra.
- Fuente de alimentación.

Diseño del soporte para la cámara



7. División de tareas

Modelo de inteligencia artificial para el reconocimiento de dígitos numéricos manuscritos e informe técnico:

> **Uriel Ravelli**

Código RAPID para dibujar números en una pizarra con el brazo robótico:

> **Sebastian Leguizamon**

Código RAPID principal para el funcionamiento de todo el proceso del sistema:

> **Sebastian Leguizamon y Manuel Sosa**

Conexión controlador - python a través de Modbus TCP IP:

> **Sebastian Leguizamon y Manuel Sosa**

Diseño e impresión de soporte acoplable al efector del robot para la cámara:

> **Rodrigo Murua**

Conexión de botones físicos para inicio y parada de emergencia:

> **Rodrigo Murua**

8. Conclusiones

El presente trabajo final representa una culminación de mucha investigación, diseño y desarrollo, en los cuales hemos trabajado en equipo de manera colaborativa y complementaria. Si bien el prototipo final aún presenta oportunidades de mejora, consideramos que hemos logrado un avance significativo en el campo de la robótica y la inteligencia artificial.

Este proyecto nos ha permitido consolidar nuestros conocimientos en programación orientada a la automatización de robots y desarrollar habilidades clave como la resolución de problemas, la toma de decisiones y la gestión de proyectos.

Agradecemos a nuestros profesores por su guía a lo largo de este proceso. 😊

9. Fuentes de investigación

> Redes neuronales artificiales:

- ▶ Cómo funcionan las redes neuronales - Inteligencia Artificial ,
- ▶ Watching Neural Networks Learn

> Clasificación de imágenes con redes neuronales artificiales:

- ▶ Crea un clasificador de perros y gatos con IA, Python y Tensorflow - Proyec...

> Redes neuronales convolucionales:

- ▶ Redes Neuronales Convolucionales - Clasificación avanzada de imágenes c...

> Deep learning:

- ▶ But what is a neural network? | Deep learning chapter 1 ,
- ▶ Gradient descent, how neural networks learn | DL2